

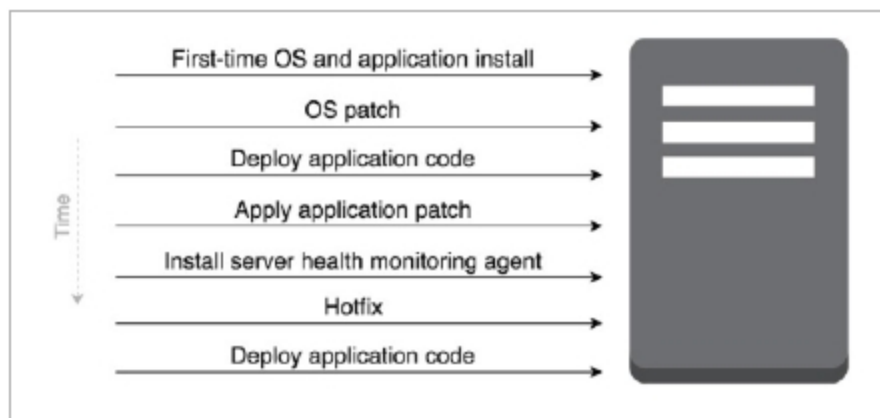


Figure 1: Mutable infrastructure

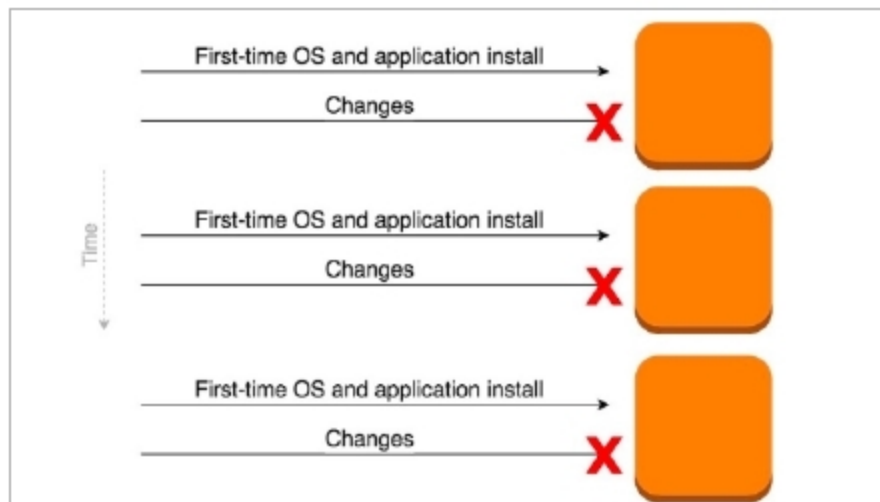


Figure 2: Immutable infrastructure

Infrastructure management with HashiCorp Terraform

HashiCorp Terraform enables the user to safely and predictably create, change and improve infrastructure. It is an open source tool that codifies APIs into declarative configuration files that can be shared amongst team members, treated as code, edited, reviewed and versioned. Terraform supports storing the state of the infrastructure, which helps to prevent configuration drift. The state can be stored in the local environment or from remote key-value or object stores. The syntax of Terraform configuration is called HashiCorp Configuration Language (HCL).

Terraform can also use JSON configurations. It supports multiple providers for orchestration. The majority of code is written in GoLang. Terraform follows clear syntax to define the resources and supports the most common data structures such as *list*, *map* and *string* to define the variables. It is quite simple to organise the code. We can read the credentials from the environment variables instead of defining them inside the Terraform configuration file. Many open source IDEs support the development of Terraform modules. We can extend the functionality of Terraform by writing custom plugins and provisioners to run the script from Bash, Ruby, Chef, etc. Reusable Terraform modules for various providers are available in the Terraform registry. Terraform Enterprise offers a Web interface to manage Terraform and the state.

Benefits of using Terraform

1. It defines Infrastructure as Code to increase operator productivity and transparency.

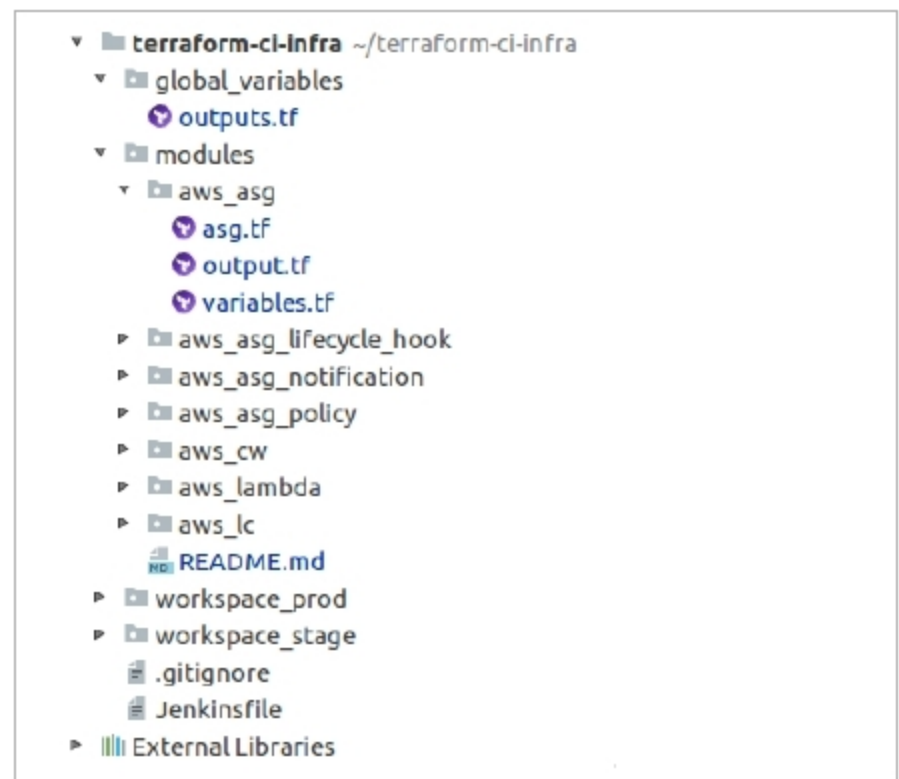


Figure 3: Terraform project structure

2. Terraform configuration can be stored in version control, shared and collaborated on by teams of operators.
3. It can track the complete history of infrastructure versions. The Terraform state can be stored on the local disk as well as on any one of the supported remote back-ends such as AWS S3, OpenStack Swift, Azure Blob, Consul, etc.
4. Terraform provides an elegant user experience for operators to safely and predictably make changes to infrastructure.
5. It builds a dependency graph from the configurations and uses this graph to generate plans, refresh state and more.
6. It separates plans and applies, and reduces mistakes and uncertainty at scale. Plans show operators what will happen, and applies execute the changes.
7. Terraform can be used to create resources across all major infrastructure providers (AWS, GCP, Azure, OpenStack, VMware and more) and third party tools such as GitHub, BitBucket, NewRelic, Consul, Docker, etc.
8. Terraform lets operators easily use the same configuration in multiple places to reduce mistakes and save time.
9. We can use the same Terraform configuration to provision identical staging, QA and production environments.
10. Common Terraform configurations can be packaged as modules, and used across teams and organisations.

```
provider "aws" {
    region = "${module.global_variables.default_aws_region}"
}

terraform {
    backend "consul" {
        address = "consul-prod.infra-as-code.com"
        scheme  = "http"
        path    = "prod/terraform/devops/myapp"
    }
}
```

Figure 4: Terraform provider and back-end configuration